

COURSE SYLLABUS

CS422 – Software Analysis and Design

1. GENERAL INFORMATION

Course name:	Software Analysis & Design
Course name (in Vietnamese):	
Course ID:	CS422
Knowledge block:	
Number of credits:	4
Credit hours for theory:	45
Credit hours for practice:	30
Credit hours for self-study:	90
Prerequisite:	
Prior-course:	
Instructors:	Associate Professor Trần Minh Triết

2. COURSE DESCRIPTION

This course introduces the common principles to analyze and design software from software requirements.

The content of this course focuses on object oriented techniques (using UML) to analyze, to design architecture, interface, business logic, and data.

Several advanced topics can be optionally introduced (e.g. design patterns, service oriented architecture...)

3. COURSE GOALS

- ❖ This course aims to equip students with basic skills to analyze and design software.
- ❖ Upon completion this course, students can:

- Describe the common principles to analyze and design software from software requirements
- Apply object oriented methods and techniques to analyze and design software
- Recognize, analyze and evaluate basic pros and cons of an existing analysis or design model, the architecture of a software system, the communication between components in a given system.
- Apply basic object oriented techniques to optimize analysis/design models to enhance the evolution and flexibility of software systems.

4. TEACHING PLAN

This teaching plan is only for reference and is subject to change.

Part 1: Overview and Revision

- ❖ Concepts in Software Engineering
 - Software
 - Software Quality
 - Software Engineering
 - Software Processes
 - Object-oriented Software Engineering
- ❖ Best Practices in Software Development
 - Develop Iteratively
 - Manage Requirements
 - Use Component Architectures
 - Model Visually (UML)
 - Continuously Verify Quality
 - Manage Change
- ❖ Concepts in Object-Oriented Programming
 - Classes and Objects:
 - Attributes and Methods
 - Visibility: Public, Protected, Private
 - Inheritance, Polymorphism
 - Relationships:
 - Generalization
 - Association, Aggregation, Composition
 - Dependency

- Other notations

Part 2: Requirements

❖ Feasibility study and Requirements elicitation and analysis

- Organization
- Business
- IT infrastructure

❖ Requirement Classification:

- Functional Requirements
- Nonfunctional Requirements

❖ Several Techniques for Requirement Elicitation:

- Interview
- Survey
- Documentation Study
- Observation
- Joint Application Design/Development

Part 3: Requirement Modeling

❖ Requirement Modeling with UML

- Problem Statement
- Actors and Use-cases
- Use-case Model
- Use-case Diagram
- Relationships in Use-case Diagrams: generalization, <<include>>, <<extend>>
- Use-case Specification: Scenarios, Flows of Events, Alternatives, Pre-conditions, Post-conditions...
- Glossary and Supplemental Specification
- Examples

❖ Data Flow Diagram (DFD)

- Introduction to Data Flow Diagram (DFD)
- Notations
- Examples

Part 4: Use-case Analysis

❖ (Analysis) Class Diagrams

- Revisions: basic concepts in OOP (again...) and notations in UML (c.f. Part 1)
- How to enhance the capability for software evolution? (tips and tricks)
- Lots of examples!

❖ State Machine diagrams

- Concepts and notations
- Examples
- Sequence Diagrams, Communication Diagrams, and VOPCs
- Concepts and notations
- Examples

Part 5: Data(base) Design

❖ Relational Database Design

- Mapping from a class diagram to a relational database
- How to enhance the capability for software evolution? (tips and tricks)
- Lots of examples!
- XML and Semi-structured Data
- Introduction to XML
- How to store data using XML
- Comparison between relational databases and XML-based data

Part 6: Software Architecture

- ❖ Introduction to Software Architecture
- ❖ Layers and Tiers
- ❖ Some guidelines for Software Architecture Analysis and Design

Part 7: (User) Interface Design

❖ Introduction

- Layout and behavior of a (user) interface
- Some common approaches for designing user interfaces
- Some (common and easy-to-understand) notations

❖ Examples and applications

- Data Input Forms: simple object, complex object, relation
- Search Forms
- Processing Business Forms
- Reports

❖ Several Techniques to Enhance Qualities of User Interfaces

- Supplemental Information
- Supplemental Operations
- Action Acceleration
- Exception Handlers

Part 8: Miscellaneous

- ❖ Design Patterns
- ❖ Software Refactoring
- ❖ Late-binding functions
- ❖ Web services (SOAP, REST) and Service Oriented Architecture
- ❖ Model-Driven Architecture
- ❖ Mashups and Widgets
- ❖ ...

5. ASSESSMENTS

This assessment plan is only for reference and is subject to change.

- ❖ Midterm exam: 2.5 marks
- ❖ Final exam: 3.5 marks
- ❖ Assignments: 1.0 mark
 - N personal assignments ($5 \leq N \leq 10$)
 - Each assignment 1/N mark
 - Total 1.0 mark
- ❖ Final Project 3.0+ marks
 - 2 student(s) per group
 - Documentation: 1.5+ marks
 - Application: 1.5+ marks
- ❖ Advanced Topics +1.0 mark
 - TBA

6. RESOURCES

- Roger S Pressman, Roger Pressman (2004), Software Engineering: A Practitioner's Approach, McGraw-Hill Science/Engineering/Math
- Ian Sommerville (2006), Software Engineering: (Update) (8th Edition) , Addison Wesley
- Mike O'Docherty (2005), Object-Oriented Analysis and Design - Understanding System Development with UML 2.0, John Wiley & Sons
- Bernd Oestereich (2001), Developing software with UML - Object-oriented analysis and design in practice, 2nd Edition, Addison Wesley
- Grady Booch et. al. (2007), Object-oriented Analysis and Design with Applications, 3rd Edition, Addison Wesley
- Craig Larman (2001), Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and the Unified Process, 2nd Edition, Prentice Hall PTR
- Dennis de Champeaux, Douglas Lea, Penelope Faure (1993). Software Engineering - Object-Oriented System Development, Addison Wesley
- Len Bass, Paul Clements, Rick Kazman (2003), Software Architecture in Practice, 2nd edition, Addison Wesley
- Sherif M. Yacoub, Hany H. Ammar (2003), Pattern-Oriented Analysis and Design: Composing Patterns to Design Software Systems, Addison Wesley.
- Alan Shalloway, James R. Trott (2004), Design Patterns Explained – A New Perspective on Object Oriented Design, Addison Wesley

7. GENERAL REGULATIONS & POLICIES

- All students are responsible for reading and following strictly the regulations and policies of the school and university.
- Students who are absent for more than 3 theory sessions are not allowed to take the exams.
- For any kind of cheating and plagiarism, students will be graded 0 for the course. The incident is then submitted to the school and university for further review.
- Students are encouraged to form study groups to discuss on the topics. However, individual work must be done and submitted on your own.